useReducer

What is `useReducer` ?

`useReducer` is a React hook for managing state that involves multiple related values or complex state transitions. It is an alternative to `useState` and follows the same pattern as Redux.

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Parameter	Description
`reducer`	A pure function `(state, action) => newState`
`initialState`	The starting value of the state
`state`	The current state
`dispatch`	A function to send actions to the reducer

When to use `useReducer` vs `useState`

Situation	Use
Simple, independent values	`useState`
Multiple related values in one object	`useReducer`
Next state depends on previous state	`useReducer`
Multiple actions affect the same state	`useReducer`
Logic is complex enough to benefit from tests	`useReducer`

Core Concepts

1. State

An object (or value) that holds the current data.

```
type State = {
  count: number;
  step: number;
};

const initialState: State = { count: 0, step: 1 };
```

2. Action

A plain object that describes **what happened**. Always has a `type`. Optionally has a `payload` for extra data.

```
type Action =
  | { type: 'increment' }
  | { type: 'decrement' }
  | { type: 'reset' }
  | { type: 'setStep'; payload: number };
```

> The `|` creates a **discriminated union** — each action shape is distinct, giving TypeScript full type safety per case.

3. Reducer

A **pure function** that takes the current state and an action, and returns the **next state**. It never mutates state directly.

```
function reducer(state: State, action: Action): State {
  switch (action.type) {
    case 'increment':
      return { ...state, count: state.count + state.step };
    case 'decrement':
      return { ...state, count: state.count - state.step };
    case 'reset':
      return { ...state, count: 0 };
    case 'setStep':
      return { ...state, step: action.payload };
    default:
      return state;
  }
}
```

> `...state` (spread) copies all existing state fields, then you override only what changed. This keeps state **immutable**.

4. Dispatch

The function returned by `useReducer` that you call to send an action to the reducer.

```
dispatch({ type: 'increment' });
dispatch({ type: 'setStep', payload: 5 });
```

Full Example — Step Counter (TypeScript)

```
import { useReducer } from 'react';

type State = {
  count: number;
  step: number;
};

type Action =
  | { type: 'increment' }
  | { type: 'decrement' }
  | { type: 'reset' }
  | { type: 'setStep'; payload: number };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case 'increment':
      return { ...state, count: state.count + state.step };
    case 'decrement':
      return { ...state, count: state.count - state.step };
    case 'reset':
      return { ...state, count: 0 };
    case 'setStep':
      return { ...state, step: action.payload };
    default:
      return state;
  }
}
```

```
const initialState: State = { count: 0, step: 1 };

export default function StepCounter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {state.count}</p>
      <label>
        Step:
        <input
          type="number"
          value={state.step}
          onChange={(e) => dispatch({ type: 'setStep', payload: Number(e.target.value) })}
        />
      </label>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
    </div>
  );
}
```

Full Example — Todo List (TypeScript)

```
import { useReducer, useState } from 'react';

type Todo = { id: number; text: string; completed: boolean };
type State = { todos: Todo[] };
type Action =
  | { type: 'add'; payload: string }
  | { type: 'toggle'; payload: number }
  | { type: 'delete'; payload: number };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case 'add':
      return {
        todos: [...state.todos, { id: Date.now(), text: action.payload, completed: false }],
      };
    case 'toggle':
```

```
      return {
        todos: state.todos.map((t) =>
          t.id === action.payload ? { ...t, completed: !t.completed } : t
        ),
      };
    case 'delete':
      return { todos: state.todos.filter((t) => t.id !== action.payload) };
    default:
      return state;
  }
}

export default function TodoApp() {
  const [state, dispatch] = useReducer(reducer, { todos: [] });
  const [input, setInput] = useState('');

  function handleAdd() {
    if (!input.trim()) return;
    dispatch({ type: 'add', payload: input.trim() });
    setInput('');
  }

  return (
    <div>
      <input value={input} onChange={(e) => setInput(e.target.value)} placeholder="New task...
      <button onClick={handleAdd}>Add</button>
      <ul>
        {state.todos.map((todo) => (
          <li key={todo.id}>
            <span
              onClick={() => dispatch({ type: 'toggle', payload: todo.id })}
              style={{ textDecoration: todo.completed ? 'line-through' : 'none', cursor: 'poi
            >
              {todo.text}
            </span>
            <button onClick={() => dispatch({ type: 'delete', payload: todo.id })}>✕</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Data Flow

Rules of Reducers

1. **Pure function** — no side effects (no API calls, no `console.log` that affects output)
2. **Never mutate state** — always return a new object using spread `{ ...state }`
3. **Predictable** — same inputs always produce the same output
4. **Handle unknown actions** — always include a `default: return state`

`useReducer` vs `useState` — Code Comparison

Same step counter with `useState`:

```
const [count, setCount] = useState(0);
const [step, setStep]   = useState(1);

<button onClick={() => setCount(count + step)}>+</button>
```

Same step counter with `useReducer`:

```
const [state, dispatch] = useReducer(reducer, { count: 0, step: 1 });

<button onClick={() => dispatch({ type: 'increment' })}>+</button>
```

> `useReducer` moves logic **out of the JSX** into a testable, pure function.